

Randomized Algorithms II

A Proposed Follow-Up to A. Andoni’s *Advanced Algorithms* (Columbia COMS 4232)
with explicit overlap to GPU / CUDA computation

June 2026

Abstract

Andoni’s *Advanced Algorithms* introduces the great paradigms of modern algorithm design — hashing, sketching/streaming, nearest-neighbor search, spectral graph algorithms, convex optimization (LP, gradient descent, interior point), multiplicative weights, online algorithms, and large-scale computation. It is, at heart, a course on *randomness and high-dimensional geometry as algorithmic tools*. This document proposes a second course that goes deeper into that frontier: the probabilistic toolkit that powers it, randomized numerical linear algebra, the spectral/Laplacian machinery behind today’s fastest graph algorithms, high-dimensional sampling, the lower-bound theory that says when to stop, and the parallel-computation models that connect all of it to real hardware. A recurring theme — and the source of this proposal’s “extra points” — is that a large fraction of these randomized algorithms are, in implementation, **GPU algorithms**: random projections are matrix multiplies, Langevin sampling is what diffusion models run, coresets is a parallel reduction, and the theory of massively parallel computation is the theory of the machine CUDA programs. Each topic below carries a green box making that link concrete.

Contents

1	What this sequel is for	2
2	Module A — The probabilistic toolkit: matrix concentration and chaining	2
3	Module B — Randomized numerical linear algebra (RandNLA)	2
4	Module C — Spectral sparsification, effective resistance, and Laplacian solvers	3
5	Module D — High-dimensional nearest neighbor: optimal and data-dependent	3
6	Module E — Sampling and Markov chains in high dimensions	4
7	Module F — Sketching lower bounds via communication complexity	4
8	Module G — Coresets	5
9	Module H — Differential privacy	5
10	Module I — Massively parallel computation and parallel randomized algorithms	5
11	Module J — Online learning, bandits, and the multiplicative-weights frontier	6

1 What this sequel is for

The base course teaches *paradigms*; learners leave able to recognize “sketch it,” “hash it,” “relax and round,” “run multiplicative weights.” This sequel has three goals. (1) **Depth**: replace black-box concentration inequalities with the real probabilistic machinery (matrix concentration, chaining), and replace “LSH exists” with the optimal, data-dependent constructions. (2) **Modernity**: cover results from the last decade — almost-linear-time max flow, optimal near-neighbor, log-concave sampling, differential privacy. (3) **Realization**: insist that an algorithm is not done until it maps to hardware, and show that the natural target for most of these is the GPU. Prereqs: the base course, probability, and linear algebra; the CUDA boxes assume only the companion “CUDA for Triton Programmers” notes.

2 Module A — The probabilistic toolkit: matrix concentration and chaining

Key idea. Sums of independent *random matrices* concentrate around their mean in spectral norm, just like sums of random scalars concentrate (Chernoff). This single fact is the engine under sketching, sparsification, and RandNLA.

The base course used scalar concentration (Markov, Chebyshev, Chernoff, Johnson–Lindenstrauss). The sequel opens with the matrix analogues: the **Matrix Chernoff/Bernstein** inequalities (Ahlswede–Winter, Tropp), which bound $\|\sum_i X_i - \mathbb{E} \sum_i X_i\|$ for independent PSD or bounded random matrices, and **matrix martingales** (Matrix Freedman) for adaptive constructions. We then cover **ϵ -nets and chaining** (Dudley, generic chaining) — the technique for controlling a random process over a *continuum* of test vectors at once, which is exactly what “preserve the norm of *every* vector in a subspace” requires. This module is the rigorous foundation that the rest of the course spends.

CUDA / parallel overlap. Not directly a kernel, but the payoff is: these bounds tell you the *embedding dimension* k — how many rows a random sketch needs. That k is the inner dimension of the GEMM you will run on Tensor Cores. Tighter concentration $\Rightarrow$ smaller sketch $\Rightarrow$ smaller matmul.

3 Module B — Randomized numerical linear algebra (RandNLA)

Key idea. To solve a huge linear-algebra problem, first multiply by a random matrix to shrink it (a “sketch”), then solve the small problem. Random projection turns $n \times d$ problems into $\text{poly}(d) \times d$ ones.

This is the most CUDA-relevant module and the natural successor to dimension reduction. Topics:

- **Subspace embeddings.** A random S (Gaussian, or the *sparse* CountSketch / OSNAP) with only $\tilde{O}(d/\epsilon^2)$ rows preserves $\|Ax\|$ for *all* x simultaneously: $\|SAx\| = (1 \pm \epsilon)\|Ax\|$. Proved with Module A.

- **Sketch-and-solve least squares.** Replace $\min_x \|Ax - b\|$ with $\min_x \|S(Ax - b)\|$ — solve a tiny regression, get a $(1 + \epsilon)$ solution in input-sparsity time.
- **Leverage-score sampling.** Importance-sample rows by their “leverage” (how much a row sticks out of the row space); the principled alternative to uniform sampling.
- **Randomized low-rank approximation / SVD** (Halko–Martinsson–Tropp): sketch the range with $Y = A\Omega$, orthonormalize to Q , return $A \approx QQ^\top A$; a couple of power iterations sharpen it. The standard way to get a rank- k SVD of a giant matrix.
- **Iterative sketching / preconditioning** (Blendenpik, LSRN): use the sketch to *precondition* an exact iterative solve — machine-precision answers, not just $(1 + \epsilon)$.

CUDA / parallel overlap. This module *is* GPU computing. Every primitive — the random projection $A\Omega$, the sketch SA , the orthonormalization, the power iterations — is a dense or sparse matrix multiply, i.e. cuBLAS / CUTLASS / Tensor-Core GEMM (your `tl.dot`). Randomized SVD is the standard recipe for low-rank work on GPUs because it replaces a sequential, hard-to-parallelize exact SVD with a few big parallel matmuls. CountSketch is a hashing kernel + scatter-add (your `atomicAdd` / segmented reduction). Sketch dimension from Module A = GEMM inner dimension.

4 Module C — Spectral sparsification, effective resistance, and Laplacian solvers

Key idea. Any dense graph can be approximated by a *sparse* one with nearly the same spectrum, by keeping each edge with probability proportional to its “effective resistance.” Sparse + a fast Laplacian solver = today’s fastest graph algorithms.

The deep successor to “spectral graph algorithms.” **Spectral sparsification** (Spielman–Srivastava): sample $\tilde{O}(n/\epsilon^2)$ edges with probability $\propto$ effective resistance (which is exactly the statistical leverage score of the edge–vertex incidence matrix — tying back to Module B), and the sparsifier preserves the Laplacian quadratic form to $(1 \pm \epsilon)$, proved by Matrix Chernoff (Module A). **Near-linear-time Laplacian / SDD solvers** (Spielman–Teng and successors) then solve $Lx = b$ in $\tilde{O}(m)$ time. **Electrical flows** interpret $Lx = b$ as currents in a resistor network and power a line of **max-flow** algorithms — from Christiano et al. through the 2022 *almost-linear-time* max-flow breakthrough (Chen–Kyng–Liu–Peng–Probst–Sachdeva), one of the headline modern results worth a full lecture.

CUDA / parallel overlap. Sparse linear algebra and graph processing are mature GPU domains: cuSPARSE, preconditioned conjugate gradient, and graph frameworks (Gunrock) run Laplacian solves and BFS/SSSP/connected-components at scale. Effective-resistance sampling is embarrassingly parallel given a solver; random-walk and PageRank-style iterations are sparse mat-vecs (SpMV) — the bandwidth-bound kernels from the CUDA course’s performance module. This is where “memory coalescing for irregular access” really bites.

5 Module D — High-dimensional nearest neighbor: optimal and data-dependent

Key idea. Plain LSH is not optimal. Letting the hash functions *adapt to the data* achieves the best possible query exponent, and “locality-sensitive filtering” pushes the time–space trade-off further.

Andoni’s own research frontier, and the natural sequel to the base LSH lecture. **Data-dependent hashing** (Andoni–Razenshteyn): the optimal exponent $\rho = \frac{1}{2c-1}$ for c -approximate near neighbor under Euclidean distance, beating data-oblivious LSH by exploiting structure in the point set. **Locality-sensitive filtering** (LSF): instead of hashing to buckets, assign points to overlapping filters, trading query time against space along a tunable curve. We also cover the **practical** side — graph-based ANN (HNSW) and quantization — and why theory and practice diverge.

CUDA / parallel overlap. Approximate nearest neighbor is a flagship GPU workload: FAISS-GPU, batched brute force, and IVF/PQ indexes power billion-scale retrieval and RAG. The core inner loop — distances from one query to many candidates — is a GEMM (query $\times$ database^T) plus a top- k reduction, i.e. Tensor-Core matmul + your warp-shuffle top- k . Hash-bucket and filter construction are parallel hashing/scatter kernels.

6 Module E — Sampling and Markov chains in high dimensions

Key idea. To sample from a complicated high-dimensional distribution, run a random walk that has it as the stationary distribution; the run-time is governed by how fast the walk “mixes” (its spectral gap / conductance).

A frontier the base course only touched. **Mixing-time theory:** conductance, spectral gap, and isoperimetry (Lovász–Simonovits; the KLS conjecture and its recent near-resolution). **Log-concave sampling:** the ball walk, hit-and-run, the Dikin walk, and the continuous-time view via **Langevin dynamics** (ULA / MALA) and **Hamiltonian Monte Carlo**. **Determinantal point processes** (DPPs) for sampling “diverse” subsets, with a beautiful link back to leverage scores and volume sampling (Module B). Applications: volume/permanent estimation, Bayesian inference, and the sampling view of optimization.

CUDA / parallel overlap. Langevin sampling — iterating $x \leftarrow x - \eta \nabla f(x) + \sqrt{2\eta} \xi$ — is precisely the loop a **diffusion model** runs at inference (cf. the diffusion-LM report): a GPU gradient/score evaluation plus Gaussian noise per step, batched over thousands of chains. The randomness needs **parallel RNG** — counter-based generators (Philox in cuRAND) that produce independent streams per thread without shared state, a gem of randomized-algorithm $\times$ CUDA design. MCMC parallelizes across chains trivially; the hard part is per-step throughput, a pure kernel-optimization problem.

7 Module F — Sketching lower bounds via communication complexity

Key idea. To prove a sketch *can’t* be smaller, show that a tiny sketch would let two far-apart players solve a communication problem known to require many bits.

The base course shows how small a sketch can be; this module shows *why it can’t be smaller* — the other half of the science. We develop **communication complexity** (deterministic, randomized, one-way), **information complexity**, and the canonical hard problems — INDEX, DISJOINTNESS, GAP-HAMMING — then use reductions (e.g. Indyk–Woodruff) to prove space lower bounds for streaming distinct-elements and frequency moments. This is the most purely theoretical module and the intellectual counterweight to the “just sketch it” reflex.

CUDA / parallel overlap. Mostly theory, but the moral is practical and parallel-relevant: lower bounds on *communication* are exactly what bound a distributed/multi-GPU algorithm. The bytes two players must exchange become the **all-reduce** / NVLink traffic between GPUs — the thing that actually limits large-scale training. Communication complexity is the theory of that bottleneck.

8 Module G — Coresets

Key idea. Compress a giant dataset into a small *weighted* subset on which any model in a class scores almost the same as on the full data — then run your expensive algorithm on the small set.

Sensitivity sampling (Feldman–Langberg): sample points with probability proportional to their worst-case “importance” over the query class, yielding coresets for k -means/median, regression, and more, of size independent of n . **Merge-and-reduce** composes small coresets into a coreset of a stream or of shards — the bridge to streaming and parallel settings.

CUDA / parallel overlap. Merge-and-reduce is a **parallel reduction tree** — structurally identical to the array-sum kernel in the CUDA course, but the “+” is “merge two coresets and re-compress.” Coreset construction (compute sensitivities, sample) is a parallel scan/sample over the data. This makes expensive clustering tractable on GPUs by shrinking the working set first.

9 Module H — Differential privacy

Key idea. Add carefully calibrated random noise so that any single person’s data barely changes the output — provable privacy as a randomized-algorithm guarantee.

The randomized-algorithms view of privacy. **Mechanisms:** Laplace and Gaussian noise calibrated to *sensitivity*; the exponential mechanism. **Composition:** how privacy loss accumulates (basic and advanced composition, Rényi-DP). **Amplification by subsampling/shuffling:** touching a random subset improves privacy “for free.” Capstone: **DP-SGD** (Abadi et al.) — per-example gradient clipping plus Gaussian noise — the workhorse of private deep learning, analyzed with the moments accountant.

CUDA / parallel overlap. DP-SGD is a GPU training algorithm: *per-example* gradient clipping breaks the usual batched-gradient trick and needs special kernels (per-sample norms, vmapped grads), and the privacy amplification relies on the *random* minibatch — again counter-based parallel RNG. A direct case where the randomized-algorithm guarantee dictates the kernel design.

10 Module I — Massively parallel computation and parallel randomized algorithms

Key idea. Model the real machine: data split across many workers with limited local memory, computing in synchronous rounds. Minimize *rounds*. This is the theory of the hardware your CUDA code runs on.

The rigorous successor to “large-scale computation models.” The **MPC** model (Karloff–Suri–Vassilvitskii and refinements) formalizes MapReduce-style computation; the goal is $O(1)$ or $O(\log \log n)$ **round complexity** for problems like connectivity, maximal independent set, matching, and sorting, often via **randomized** sparsification and sampling. We also revisit the classical **PRAM** primitives — parallel *prefix-sum* (*scan*), reduction, and sort — that are the atoms of all of it, and the “1-vs-2-cycle” conjecture that frames MPC lower bounds.

CUDA / parallel overlap. This is the explicit theory $\leftrightarrow$ CUDA bridge. PRAM scan/reduce/sort are exactly the CUB / Thrust primitives every GPU programmer uses; “round complexity” across MPC workers is “kernel launches + **all-reduces**” across GPUs. Work-efficiency and depth (the PRAM cost measures) are how you reason about a parallel kernel’s quality. If you understand why a parallel scan is $O(\log n)$ depth, you understand both the theory and the CUB implementation.

11 Module J — Online learning, bandits, and the multiplicative-weights frontier

Key idea. Make decisions one at a time against an adversary and still do almost as well as the best fixed choice in hindsight — “no regret.” Multiplicative weights is the tip of a large iceberg.

Deepening the base MWU/online module. **Online convex optimization** (online gradient descent, regret bounds), **adversarial bandits** (EXP3) and **stochastic bandits** (UCB) with $\tilde{O}(\sqrt{T})$ regret, and the unifying **Follow-the-Regularized-Leader** view. Connections: MWU as a master algorithm for solving LPs/zero-sum games (back to optimization), and online-to-batch conversion linking regret to generalization.

CUDA / parallel overlap. Bandit/online updates are lightweight, but the settings that need them — large-scale recommendation, hyperparameter/ neural-architecture search, RLHF-style exploration — run on GPU fleets, where thousands of arms/contexts are evaluated in parallel batches. The exploration noise is, once more, parallel RNG.

12 Synthesis: the randomized-algorithm $\leftrightarrow$ CUDA map

Topic	Core randomized idea	CUDA realization
RandNLA (B)	random projection / sketch	Tensor-Core GEMM (cuBLAS/CUTLASS), randomized SVD
Spectral / Laplacian (C)	eff.-resistance sampling	SpMV, PCG, cuSPARSE, Gunrock
Optimal ANN (D)	data-dependent hashing	FAISS-GPU: GEMM + warp top- k
Sampling / MCMC (E)	Langevin / mixing	diffusion-model loop; Philox parallel RNG
Sketching bounds (F)	communication complexity	all-reduce / NVLink traffic limits
Coresets (G)	sensitivity sampling	merge-and-reduce = parallel reduction tree
Differential privacy (H)	noise + subsampling	DP-SGD per-sample kernels; RNG
MPC / parallel (I)	randomized round reduction	CUB/Thrust scan-reduce-sort; multi-GPU
Online / bandits (J)	no-regret exploration	batched arm eval; RNG

Table 1: Most modules terminate in a GPU kernel; the three recurring ones are *GEMM*, *parallel reduction/scan*, and *counter-based RNG*.

A bonus thread that ties it together. A single guest lecture on **counter-based / parallel pseudorandomness** (Philox, Threefry; Salmon et al.) is the perfect capstone: it is a genuine *randomized-algorithms* object (provable statistical quality, 2^{128} independent streams) whose entire reason for existing is *CUDA* (stateless per-thread RNG with no synchronization). Every module above quietly depends on it.

13 Proposed schedule (one semester, ~13 weeks)

Weeks	Module	Anchor result
1–2	A. Probabilistic toolkit	Matrix Chernoff; chaining
3–4	B. RandNLA	subspace embeddings; randomized SVD
5–6	C. Spectral sparsification & Laplacians	almost-linear max flow
7	D. Optimal / data-dependent ANN	$\rho = 1/(2c-1)$
8–9	E. High-dimensional sampling	log-concave / Langevin mixing
10	F. Sketching lower bounds	Gap-Hamming $\Rightarrow$ streaming bounds
11	G. Coresets + H. Differential privacy	sensitivity sampling; DP-SGD
12	I. Massively parallel computation	connectivity in $O(\log \log n)$ rounds
13	J. Online & bandits + parallel-RNG capstone	EXP3/UCB; Philox

Selected references (representative, not exhaustive). Tropp, *An Introduction to Matrix Concentration Inequalities*, 2015; Woodruff, *Sketching as a Tool for Numerical Linear Algebra*, 2014; Halko, Martinsson, Tropp, *Finding Structure with Randomness* (randomized SVD), 2011; Spielman & Srivastava, *Graph Sparsification by Effective Resistances*, 2011; Chen, Kyng, Liu, Peng, Probst Gutenberg, Sachdeva, *Maximum Flow in Almost-Linear Time*, 2022; Andoni & Razenshteyn, *Optimal Data-Dependent Hashing for Approximate Near Neighbors*, 2015; Vempala, *Geometric Random Walks: A Survey*; Feldman & Langberg, *A Unified Framework for Approximating and Clustering Data* (coresets), 2011; Dwork & Roth, *The Algorithmic Foundations of Differential Privacy*, 2014; Abadi et al., *Deep Learning with Differential Privacy* (DP-SGD), 2016; Karloff, Suri, Vassilvitskii, *A Model of Computation for MapReduce*, 2010; Salmon et al., *Parallel Random Numbers: As Easy as 1, 2, 3* (Philox), 2011; Lattimore & Szepesvári, *Bandit Algorithms*, 2020.